Федеральное государственное бюджетное образовательное учреждение высшего образования



«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)_____

О т ч е т

по лабораторной работе № 2

Название лабораторной работы: Создание контейнеров. Исключения. Шаблоны

Дисциплина: Алгоритмизация и программирование

Студент гр. **ИУ6-22Б** _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цели работы:

1. Разработать иерархию классов для реализации контейнера типа «стек».
2. Изучить механизмы обработки исключений в C++ для обеспечения отказоустойчивости.
3. Освоить создание шаблонных классов (templates) для обеспечения универсальности контейнера.
4. Создать тестирующую программу с графическим интерфейсом на базе библиотеки Qt.

1. Теоретические сведения и обоснование структуры данных

Контейнер реализован на основе связного списка, что позволяет динамически выделять память под каждый новый элемент, обеспечивая гибкость размера стека (в пределах заданного максимума). Использование шаблонов (template <typename T>) позволило создать универсальный контейнер, способный хранить любые типы данных, что значительно упрощает сопровождение и повторное использование кода.

Ниже представлена диаграмма классов приложения, отражающая структуру иерархии:

Рисунок 1 – Диаграмма классов приложения

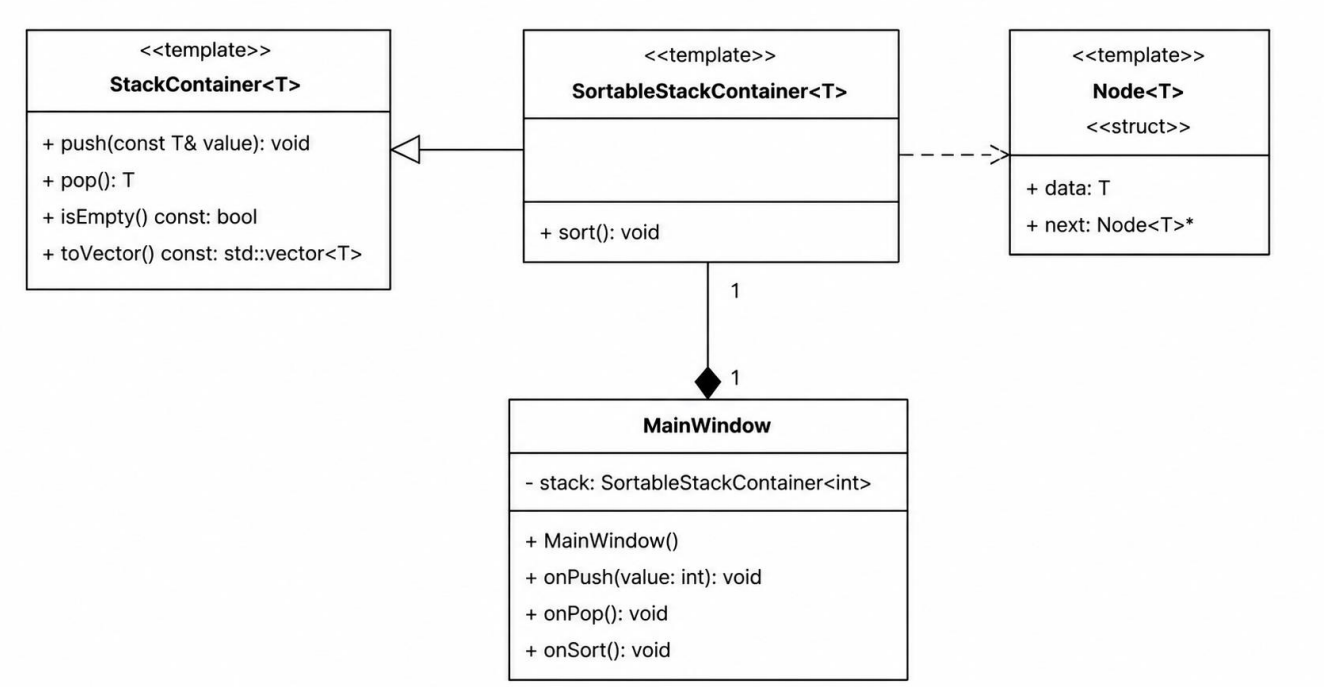


Рисунок 1 - Диаграмма классов

2. Реализация контейнера

Для обеспечения корректности работы контейнера были реализованы следующие исключения, наследуемые от std::runtime_error:

- StackOverflowException: выбрасывается при попытке добавить элемент в переполненный стек.
- StackEmptyException: выбрасывается при попытке извлечь элемент из пустого стека.

Разработанный шаблонный контейнер StackContainer<T> инкапсулирует логику управления памятью (через структуру Node<T>) и предоставляет безопасный интерфейс взаимодействия.

Класс-потомок SortableStackContainer<T> расширяет базовый функционал, добавляя методы сортировки, реализуемые через временное копирование данных в QVector и использование стандартного алгоритма std::sort.

3. Тестирование и графический интерфейс

Тестирующая программа на Qt использует QMainWindow в качестве главного окна. Интерфейс включает поля ввода для целых чисел и символов, а также текстовое поле QTextEdit для отображения состояния стека. Обработка исключений осуществляется непосредственно в методах-слотах MainWindow через блоки try-catch, что предотвращает аварийное завершение программы при ошибочных действиях пользователя.

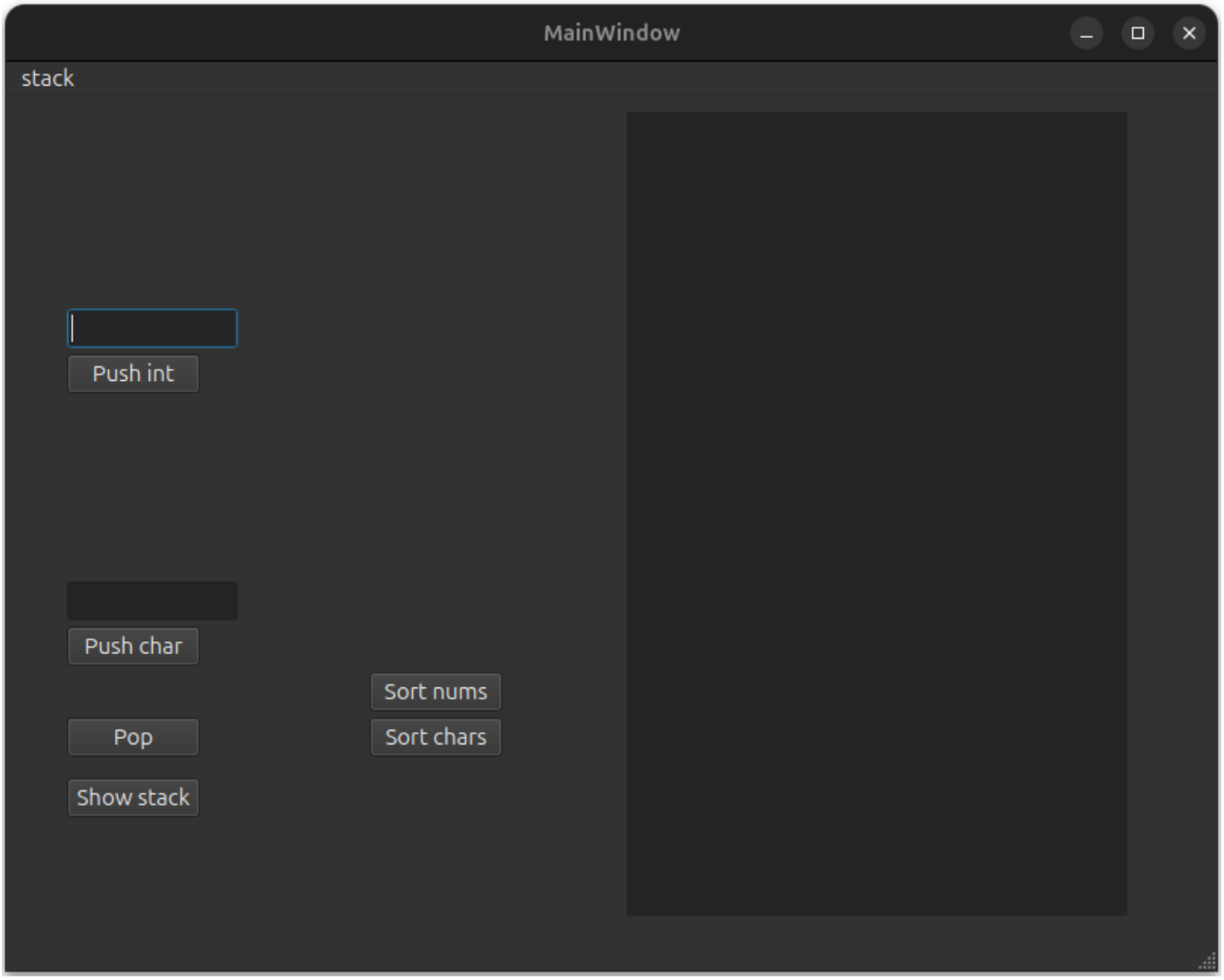


Рисунок 2 - Главное окно приложения

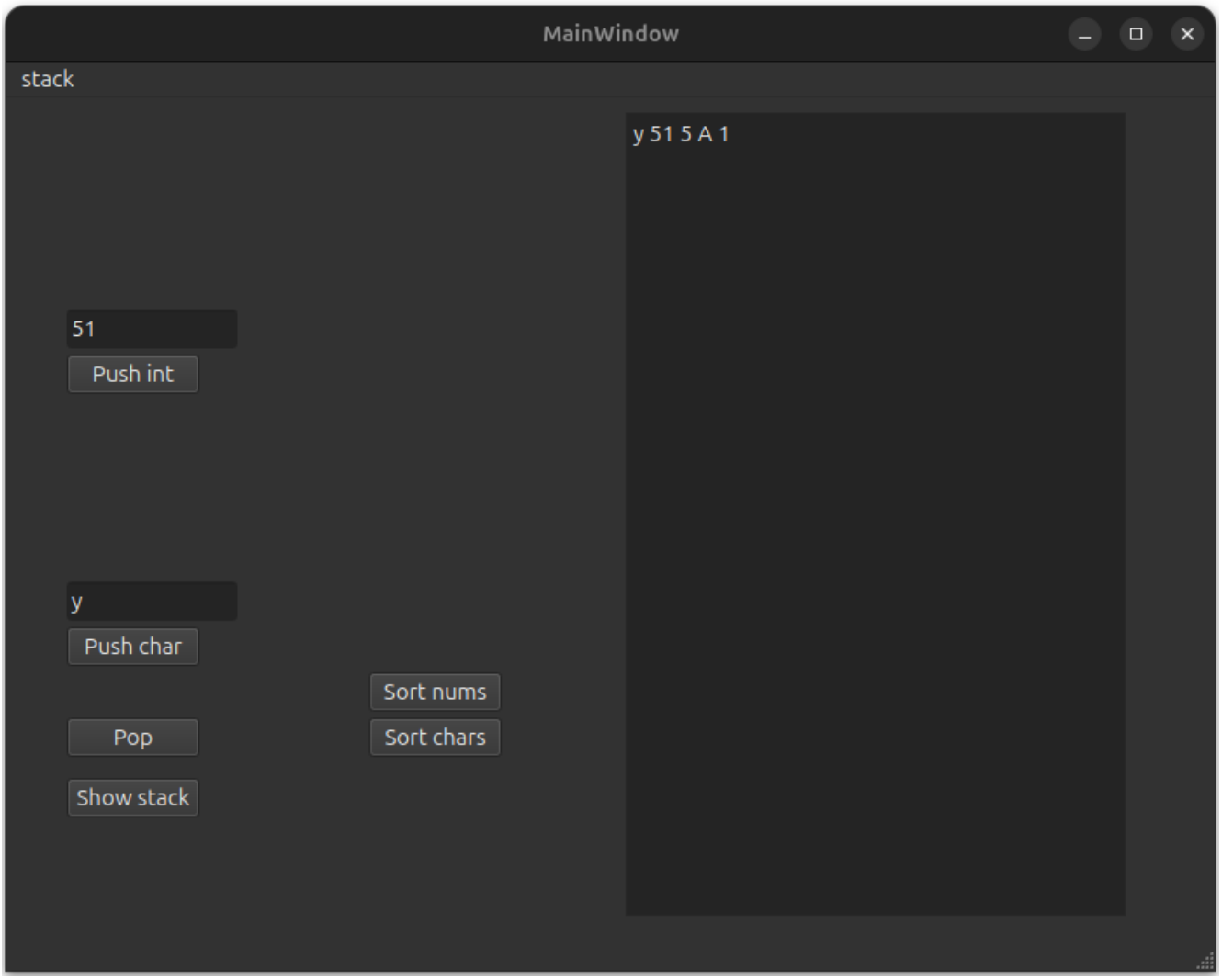


Рисунок 3 - Демонстрация работы программы

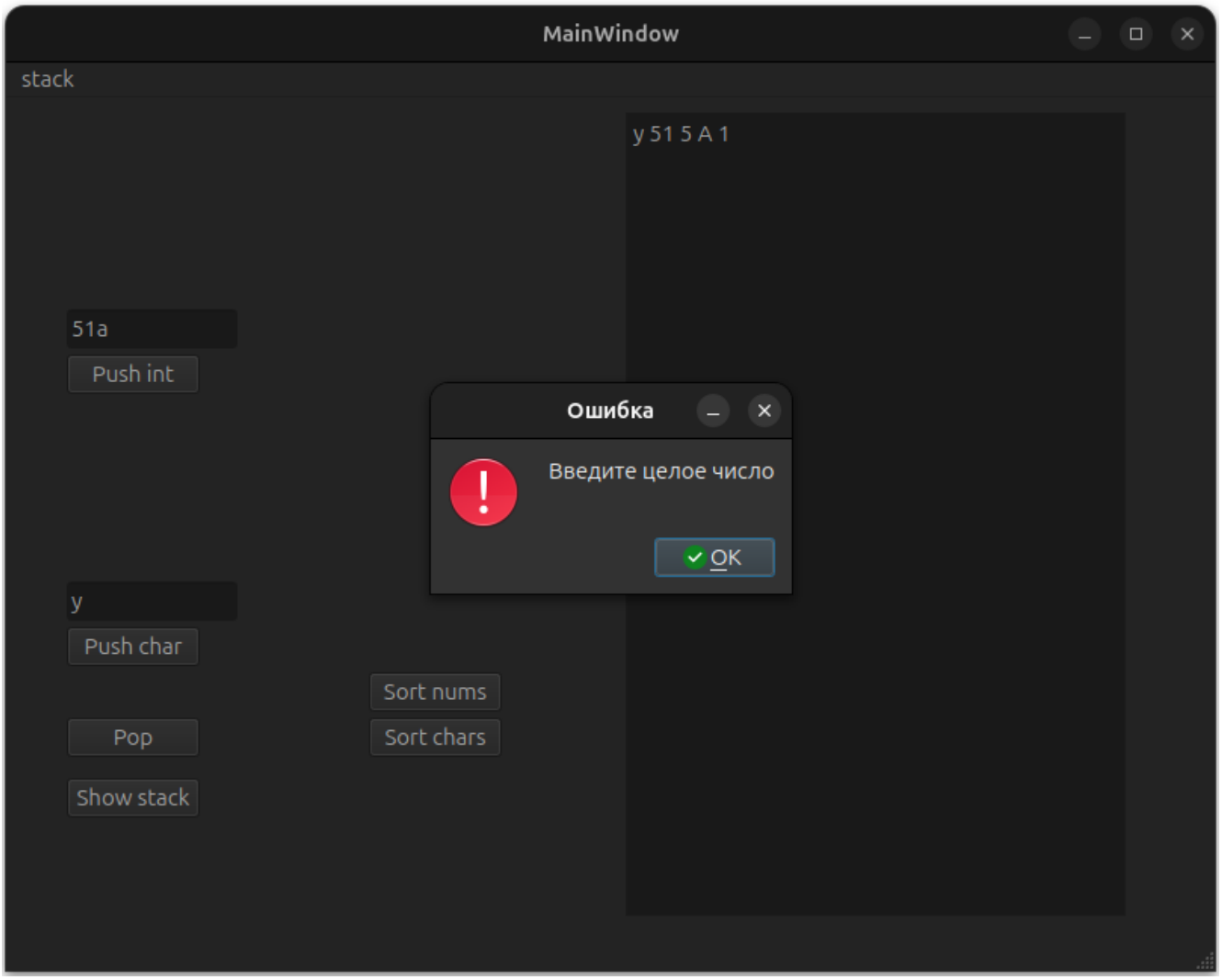


Рисунок 4 - Срабатывание исключения

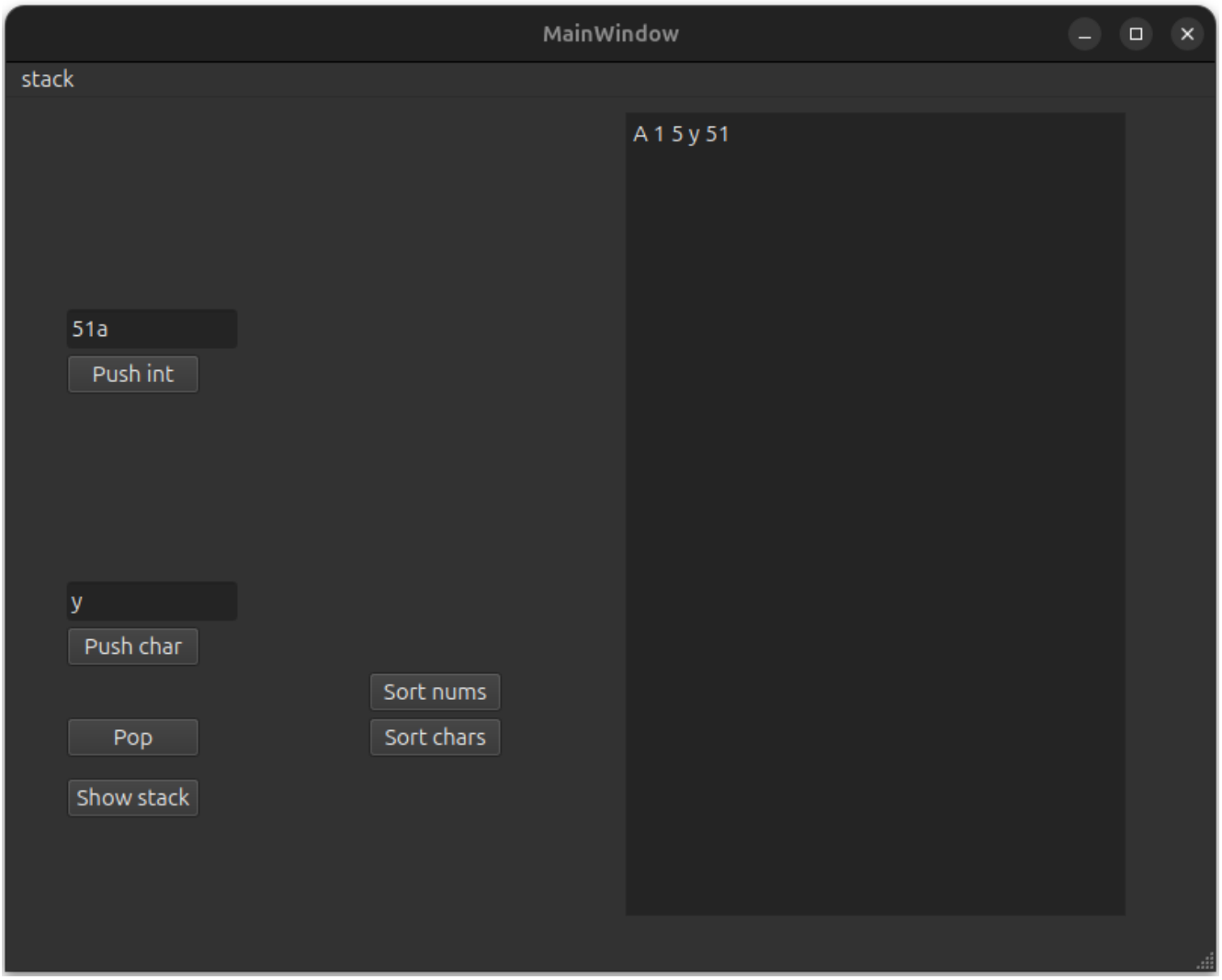


Рисунок 5 - Отсротированный стек

Вывод:

В ходе выполнения лабораторной работы была успешно разработана иерархия классов контейнера типа «стек», а также освоены механизмы обработки исключений, обеспечивающие высокую надежность приложения. Перевод классов на шаблонную реализацию позволил сделать контейнер независимым от конкретного типа данных, что соответствует принципам обобщенного программирования в C++. Графический интерфейс на базе Qt позволил наглядно визуализировать работу контейнера, упростив отладку и проверку корректности алгоритмов сортировки. В результате проделанной работы были закреплены навыки проектирования масштабируемых и универсальных программных архитектур.